Experiment No : 03

Aim :

Learn basics of maplotlib library and its use as visualisation tool in data science pipeline and learn about line plots and scatter plots.

Theory :

Matplotlib is a multi-platform data visualization library built on NumPy arrays, and designed to work with the broader SciPy stack.

It was conceived by John Hunter in 2002, originally as a patch to IPython for enabling interactive MATLAB-style plotting via gnuplot from the IPython command line.

One of Matplotlib's most important features is its ability to play well with many operating systems and graphics backends. Matplotlib supports dozens of backends and output types, which means you can count on it to work regardless of which operating system you are using or which output format you wish.

This cross-platform, everything-to-everyone approach has been one of the great strengths of Matplotlib. It has led to a large user base, which in turn has led to an active developer base and Matplotlib's powerful tools and ubiquity within the scientific Python world.

In recent years, however, the interface and style of Matplotlib have begun to show their age. Newer tools like ggplot and ggvis in the R language, along with web visualization toolkits based on D3js and HTML5 canvas, often make Matplotlib feel clunky and old-fashioned.

Performance :

[Students need to execute eaxh and every cell in this section and note the output of the same. Once done they have to answer Questions mentioned in review section]

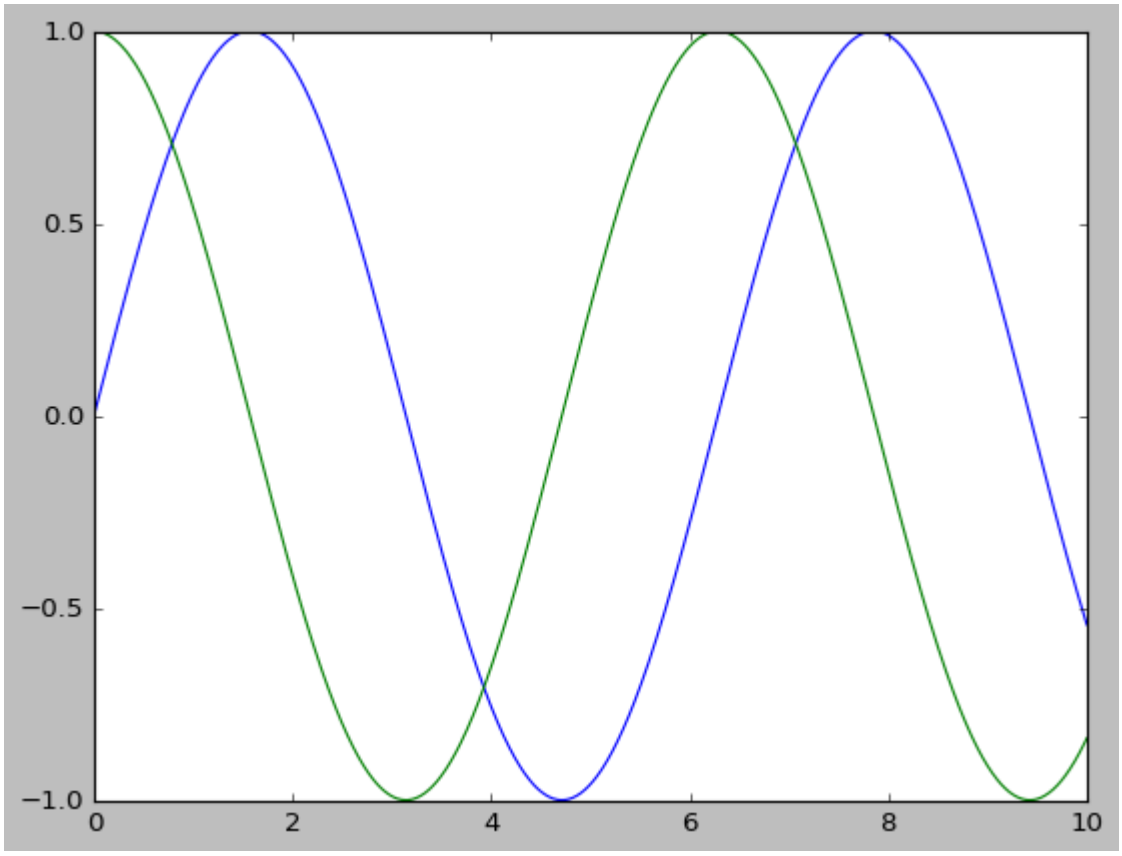
```
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.__version__
```

```
'3.10.0'
```

```
#setting ploting style
plt.style.use('classic')
```

```
%matplotlib inline
import numpy as np
x = np.linspace(0, 10, 1000)

fig = plt.figure()
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x));
```



Saving Figures to File

One nice feature of Matplotlib is the ability to save figures in a wide variety of formats. Saving a figure can be done using the `savefig()` command. For example, to save the previous figure as a PNG file, you can run this:

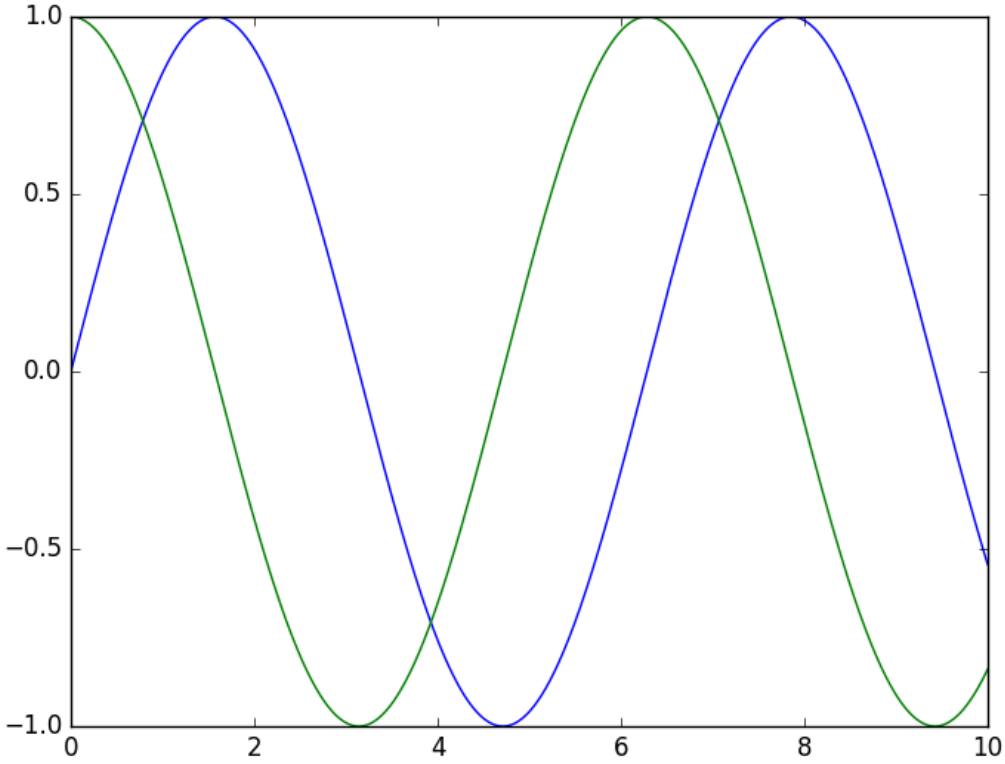
```
#Save above plot with file named my_figure.png in Current Working Directory
fig.savefig('my_figure.png')
```

```
# Cross check the folder content
!ls -lh my_figure.png
```

```
-rw-r--r-- 1 root root 45K Feb 24 13:44 my_figure.png
```

```
# use Image package in python to check contents of png file in cwd
from IPython.display import Image
```

```
Image('my_figure.png')
```




```
#checking supported file types by matplotlib
fig.canvas.get_supported_filetypes()
```

```
{'eps': 'Encapsulated Postscript',
 'jpg': 'Joint Photographic Experts Group',
 'jpeg': 'Joint Photographic Experts Group',
 'pdf': 'Portable Document Format',
 'pgf': 'PGF code for LaTeX',
 'png': 'Portable Network Graphics',
 'ps': 'Postscript',
 'raw': 'Raw RGBA bitmap',
 'rgba': 'Raw RGBA bitmap',
 'svg': 'Scalable Vector Graphics',
 'svgz': 'Scalable Vector Graphics',
 'tif': 'Tagged Image File Format',
 'tiff': 'Tagged Image File Format',
 'webp': 'WebP Image Format'}
```

Matplotlib API Interfaces

A potentially confusing feature of Matplotlib is its dual interfaces: a convenient MATLAB-style state-based interface, and a more powerful object-oriented interface. This section we will review them breifly.

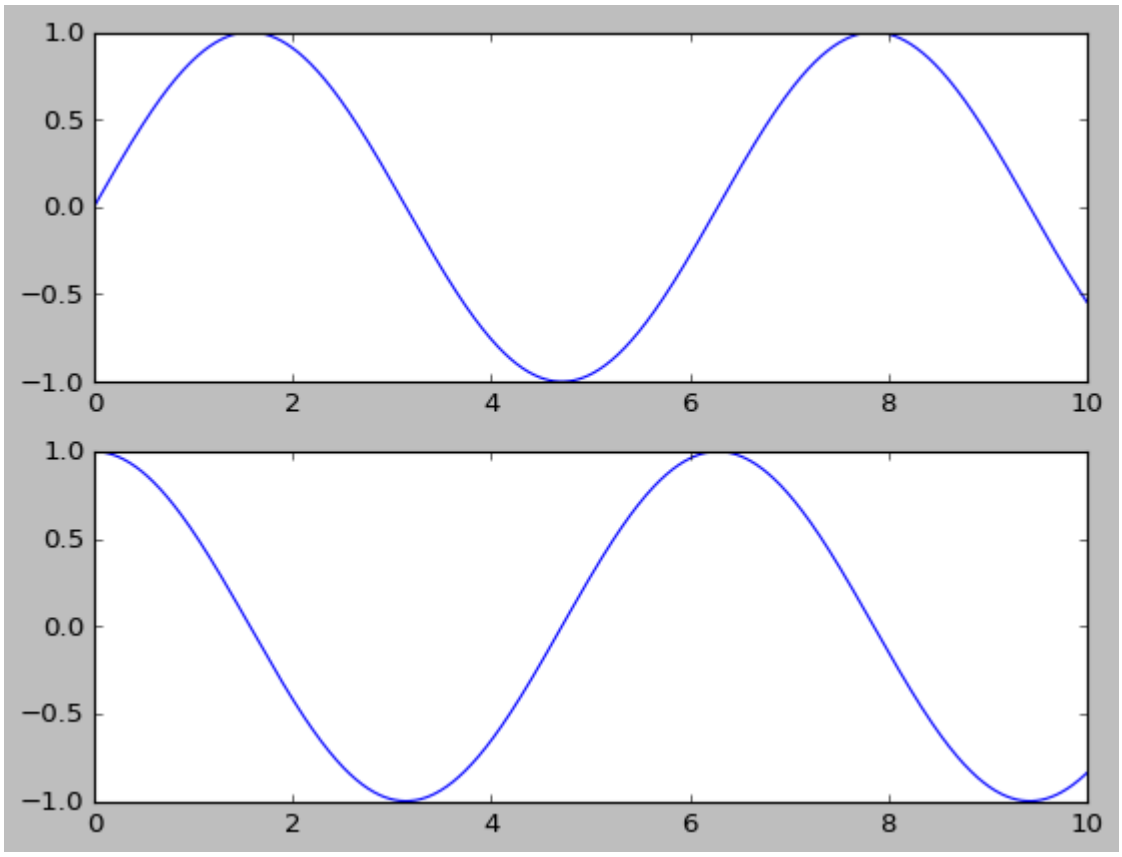
MATLAB-style Interface

Matplotlib was originally written as a Python alternative for MATLAB users, and much of its syntax reflects that fact. The MATLAB-style tools are contained in the pyplot (`plt`) interface. See the example inthe code cell below.

```
plt.figure()  # create a plot figure

# create the first of two panels and set current axis
plt.subplot(2, 1, 1) # (rows, columns, panel number)
plt.plot(x, np.sin(x))

# create the second panel and set current axis
plt.subplot(2, 1, 2)
plt.plot(x, np.cos(x));
```



This interface is *stateful*: It keeps track of the "current" figure and axes, which are where all `plt` commands are applied.

You can get a reference to these using the `plt.gcf()` (get current figure) and `plt.gca()` (get current axes) routines.

While this stateful interface is fast and convenient for simple plots, it is easy to run into problems.

For example, once the second panel is created, how can we go back and add something to the first? This interface does have a solution but it is not a easy solution.

Object-oriented interface of Matplotlib

The object-oriented interface can nicely handle more complicated plots, and for when you want more control over your figure.

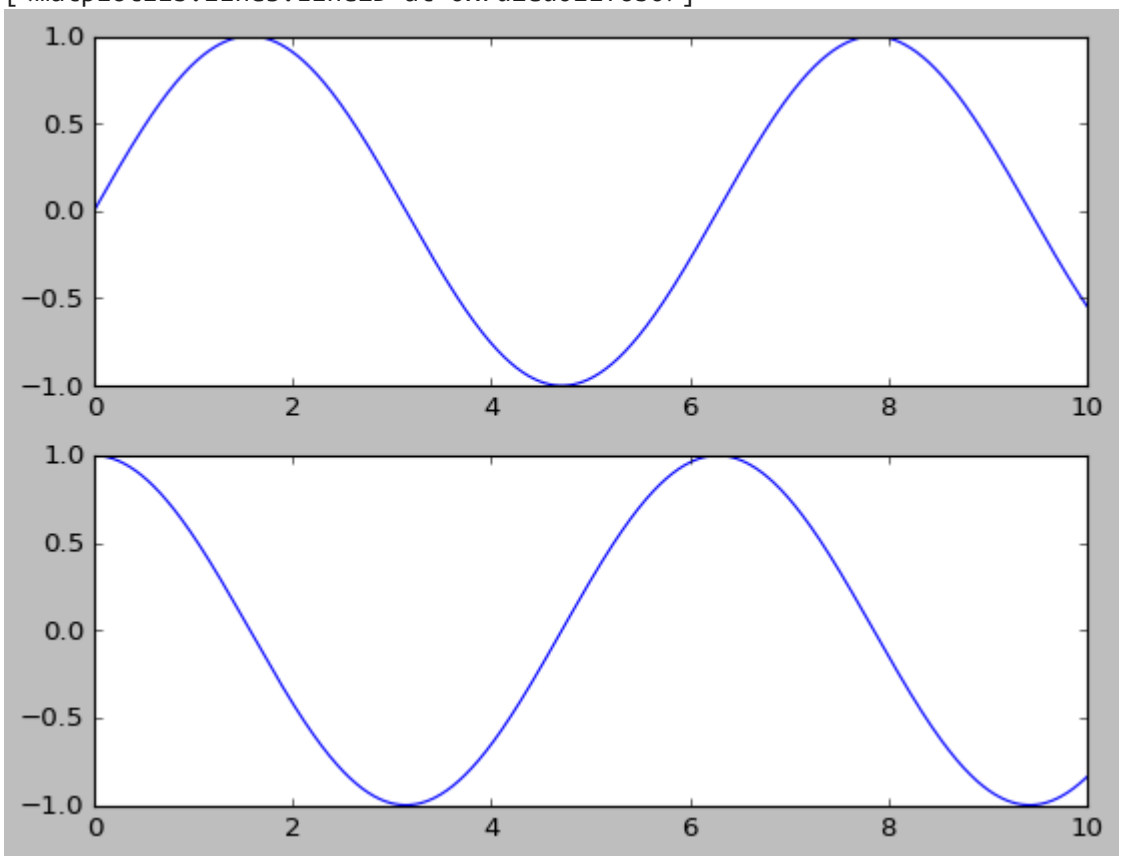
Rather than depending on some notion of an "active" figure or axes, in the object-oriented interface the plotting functions are *methods* of explicit `Figure` and `Axes` objects.

To re-create the previous plot using this style of plotting, you might do the following:

```
#First create a grid of plots
# ax will be an array of two Axes objects
fig, ax = plt.subplots(2)

# Call plot() method on the appropriate object
ax[0].plot(x, np.sin(x))
ax[1].plot(x, np.cos(x))
```

```
[<matplotlib.lines.Line2D at 0x7d2ed011f6b0>]
```



The difference is as small as switching `plt.plot()` to `ax.plot()`.

Now we start exploring different types of plots in following sections.

Simple Line Plots

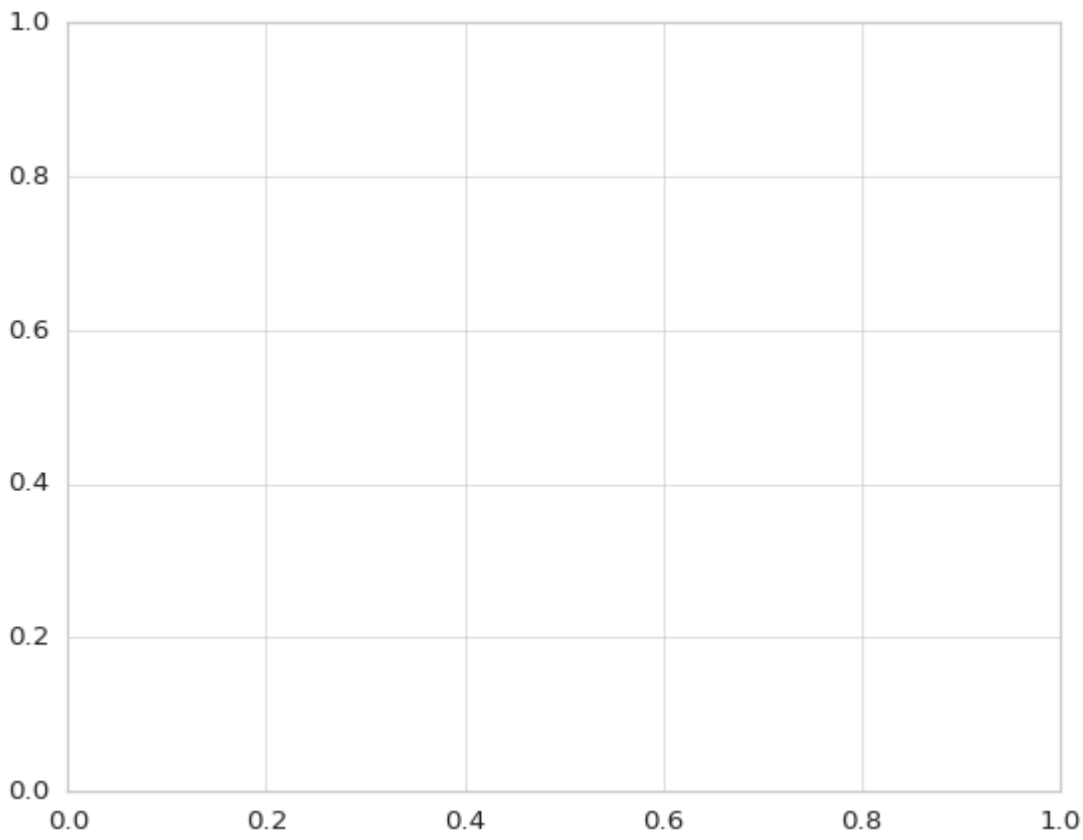
Perhaps the simplest of all plots is the visualization of a single function `y = f(x)`.

Here we will take a first look at creating a simple plot of this type.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns # Import seaborn library

#setting plotting style to seaborn's whitegrid
sns.set_style('whitegrid')
```

```
# creating objects of figure and axes.
fig = plt.figure()
ax = plt.axes()
```

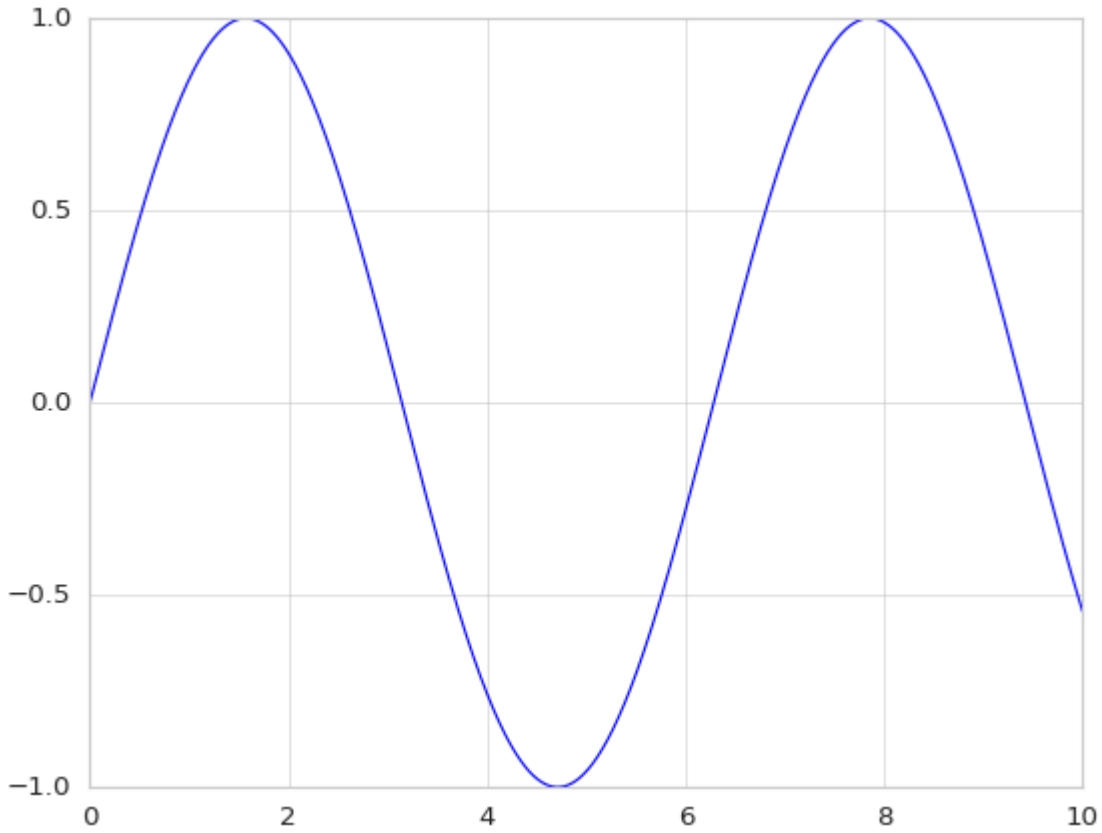


In Matplotlib, the *figure* (an instance of the class `plt.Figure`) can be thought of as a single container that contains all the objects representing axes, graphics, text, and labels.

The *axes* (an instance of the class `plt.Axes`) is what we see above: a bounding box with ticks and labels, which will eventually contain the plot elements that make up our visualization.

Once we have created an axes, we can use the `ax.plot` function to plot some data.

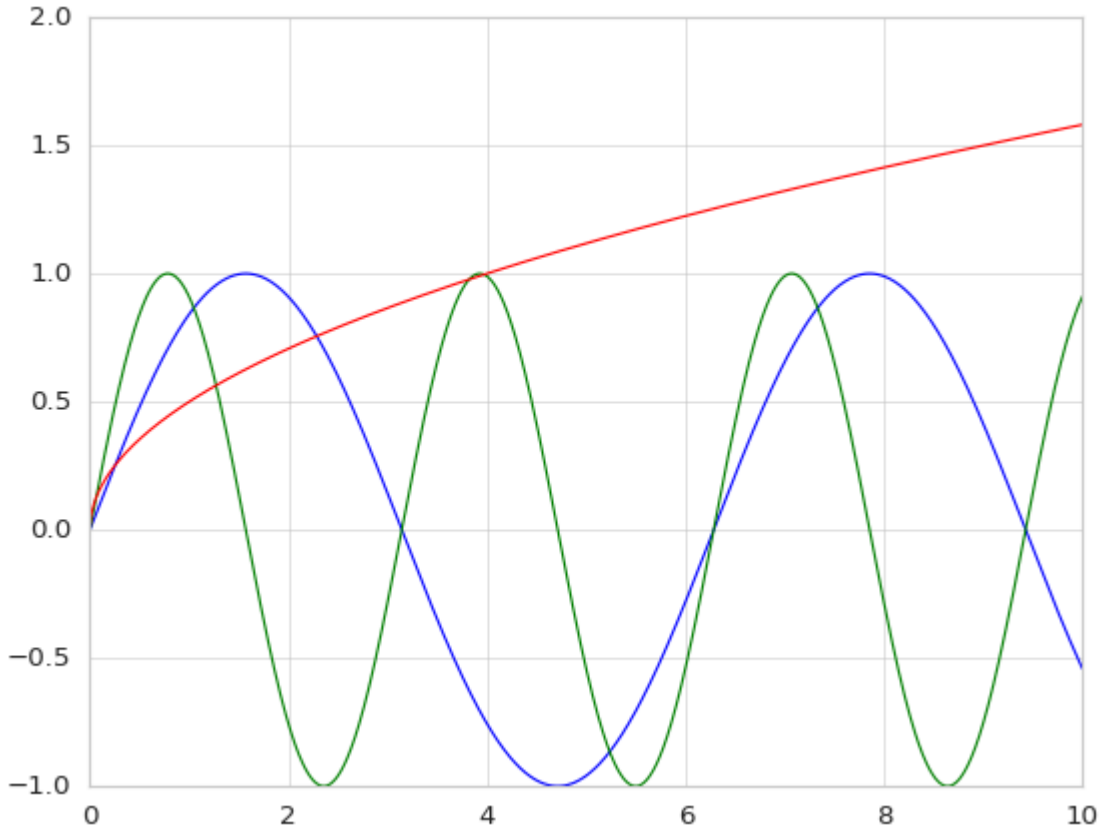
```
# we will plot sine curve using ax.plot function
fig = plt.figure()
ax = plt.axes()
x = np.linspace(0, 10, 1000)
ax.plot(x, np.sin(x));
```



The word line in line plot refers to the function we want to plot. Multiple functions can be plot on same plot like shown below.

```
plt.plot(x, np.sin(x))   #shown in blue
plt.plot(x, np.sin(2*x)) #Shown in green
plt.plot(x,(x**(1/2))/2) #Shown in red
```

```
[<matplotlib.lines.Line2D at 0x7d2ed0fa1640>]
```

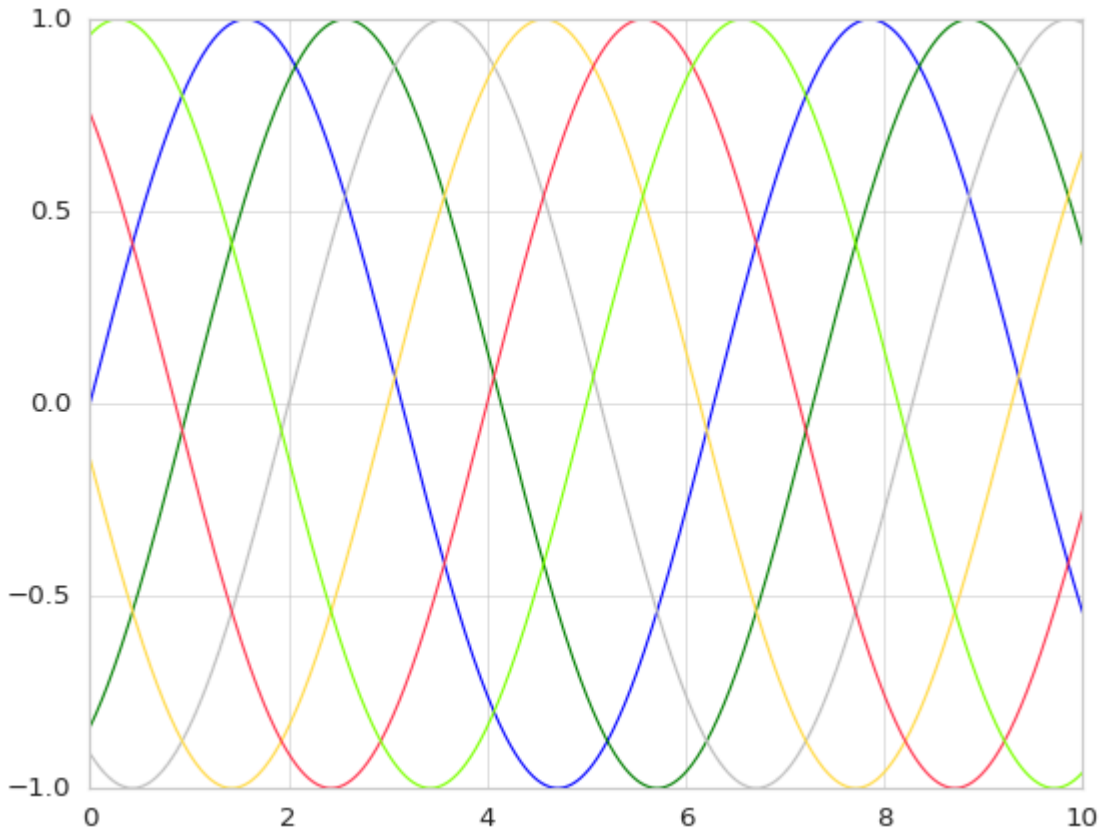


Double-click (or enter) to edit

Adjusting the Plot: Line Colors and Styles

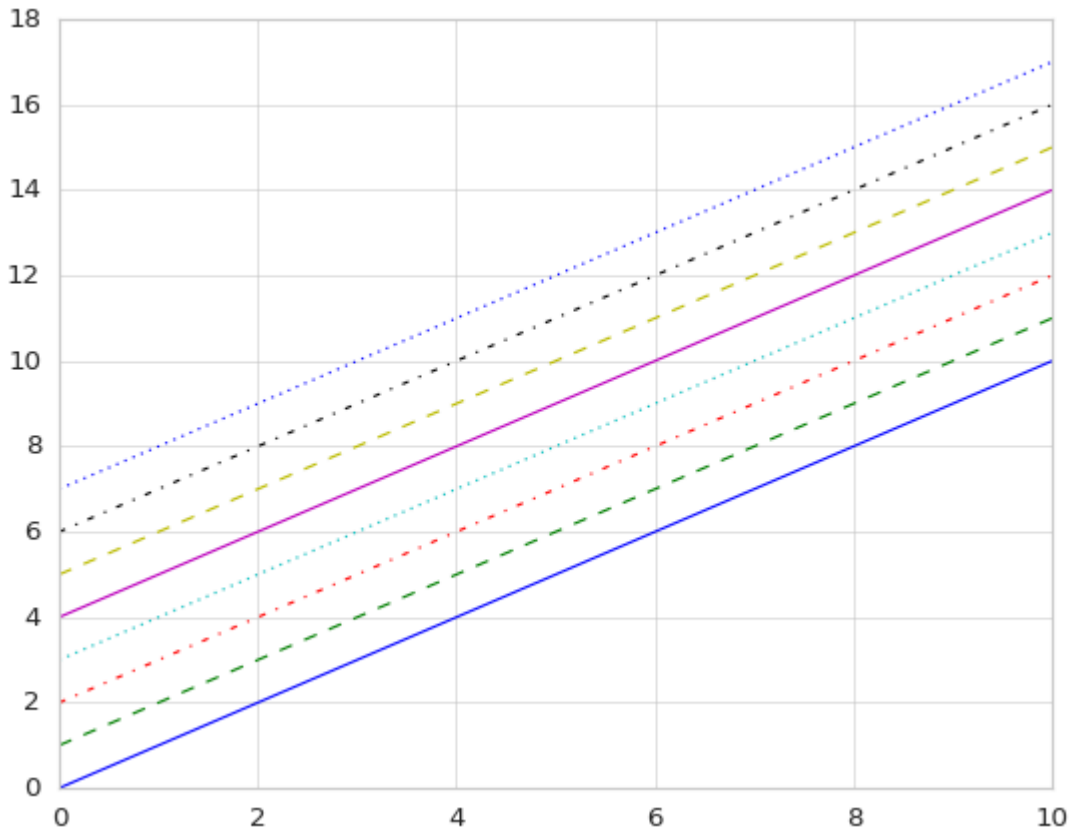
The first adjustment you might wish to make to a plot is to control the line colors and styles. The *plt.plot()* function takes additional arguments that can be used to specify these. To adjust the color, you can use the color keyword, which accepts a *string* argument representing virtually any imaginable color. The color can be specified in a variety of ways:

```
plt.plot(x, np.sin(x - 0), color='blue')        # specify color by name
plt.plot(x, np.sin(x - 1), color='g')           # short color code (rgbcmyk)
plt.plot(x, np.sin(x - 2), color='0.75')        # Grayscale between 0 and 1
plt.plot(x, np.sin(x - 3), color='#FFDD44')     # Hex code (RRGGBB from 00 t
plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) # RGB tuple, values 0 to 1
plt.plot(x, np.sin(x - 5), color='chartreuse'); # all HTML color names suppo
```




```
plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');

# For short, you can use the following codes:
plt.plot(x, x + 4, linestyle='-')  # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':');  # dotted
```

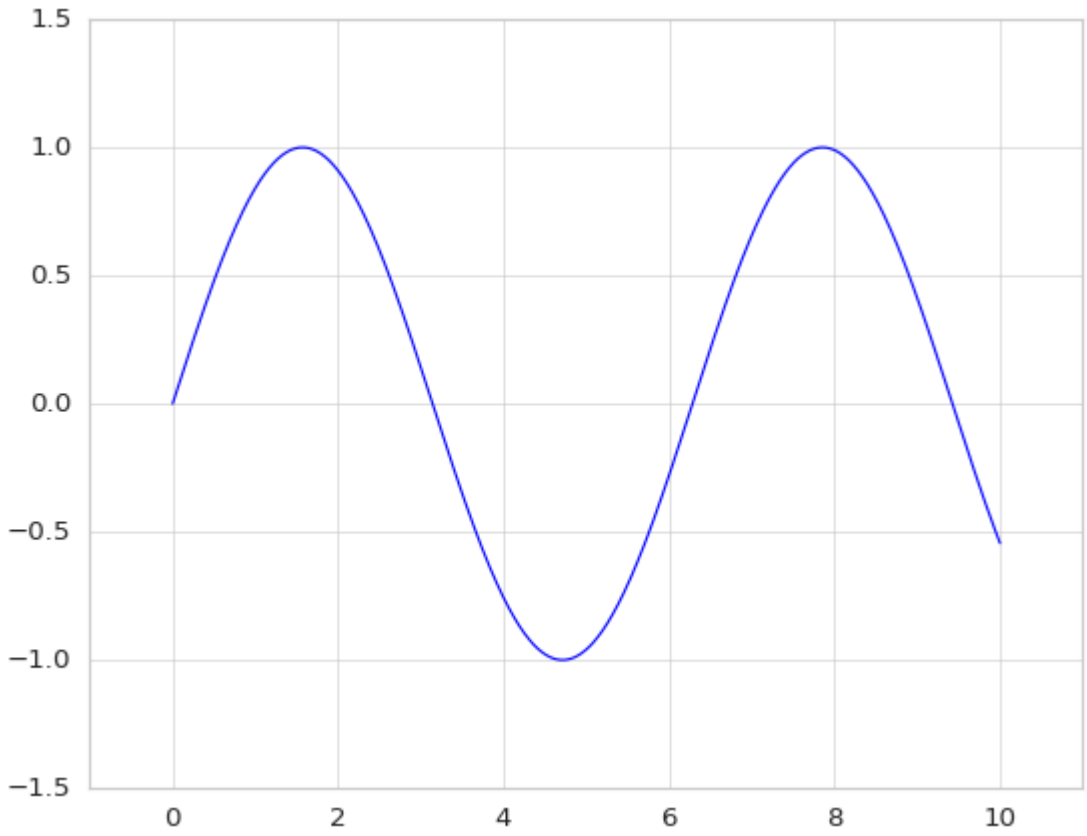


Adjusting the Plot: Axes Limits

Matplotlib does a decent job of choosing default axes limits for your plot, but sometimes it's nice to have finer control. The most basic way to adjust axis limits is to use the *plt.xlim()* and *plt.ylim()* methods:

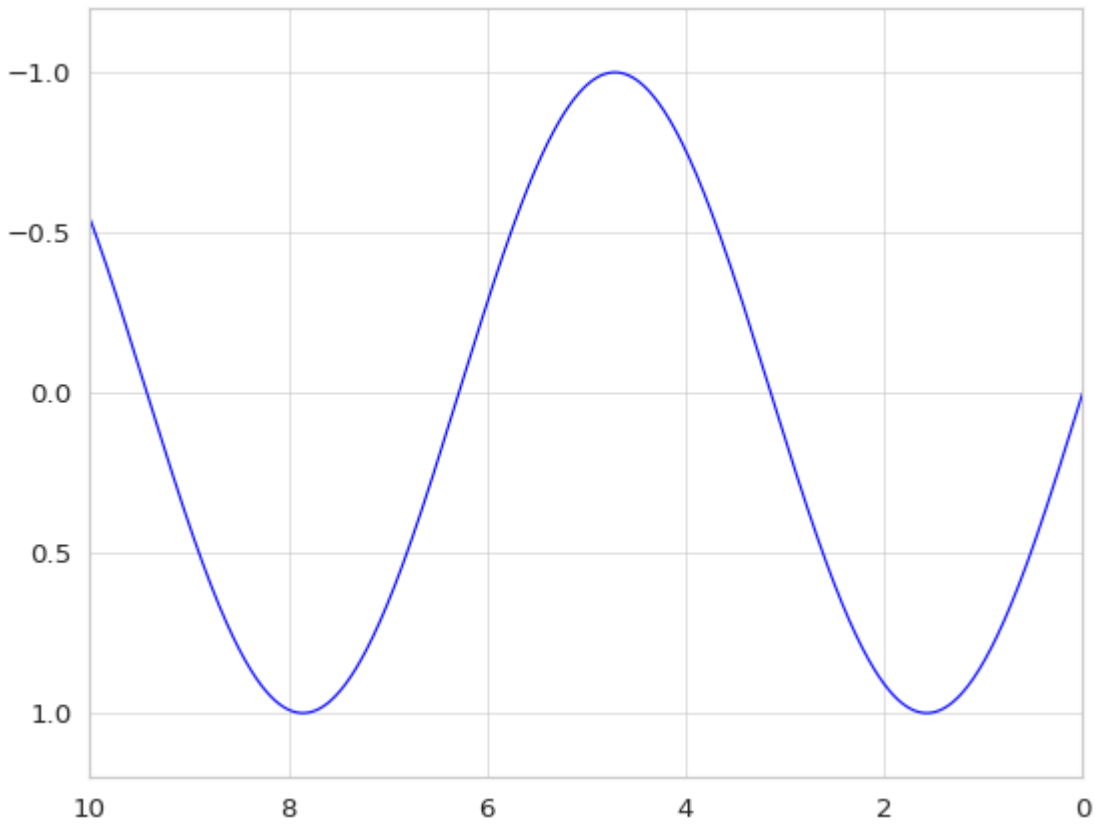
```
plt.plot(x, np.sin(x))

plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5);
```



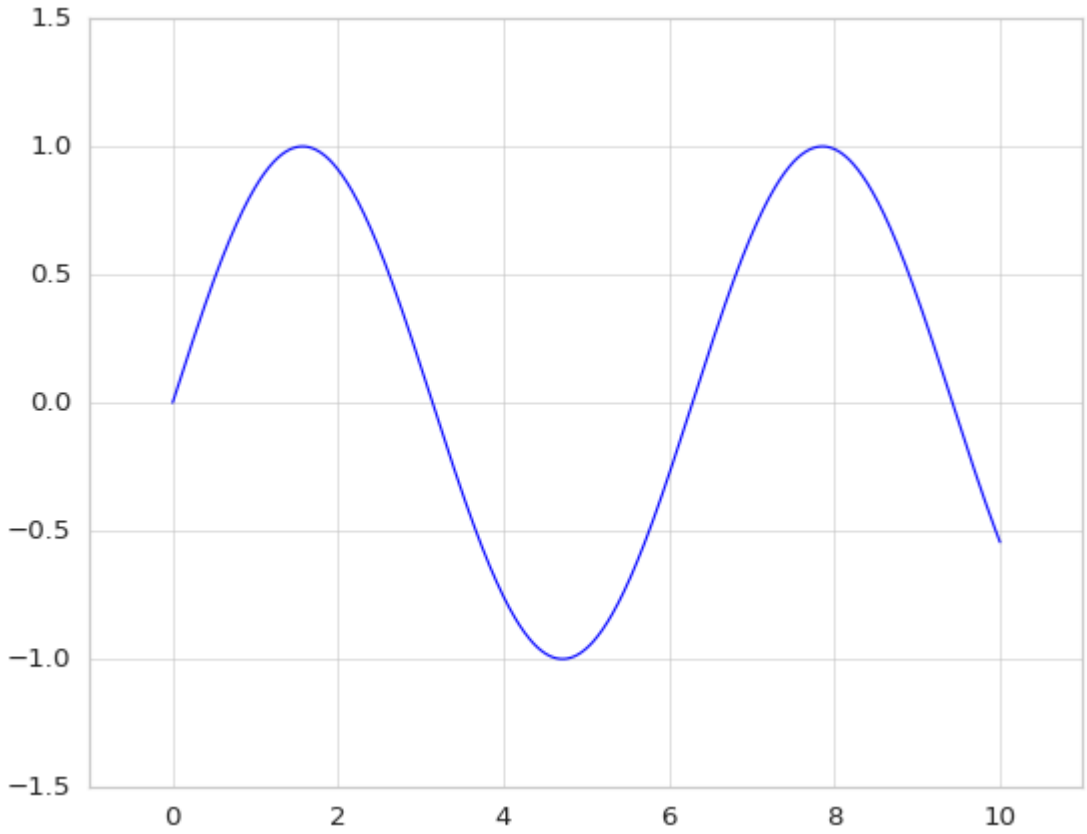

```
# Reverse the Axis Limit
plt.plot(x, np.sin(x))

plt.xlim(10, 0)
plt.ylim(1.2, -1.2);
```



A useful related method is plt.axis() (note here the potential confusion between axes with an e, and axis with an i). The plt.axis() method allows you to set the x and y limits with a single call, by passing a list which specifies **[xmin, xmax, ymin, ymax] :**

```
plt.plot(x, np.sin(x))
plt.axis([-1, 11, -1.5, 1.5]);
```

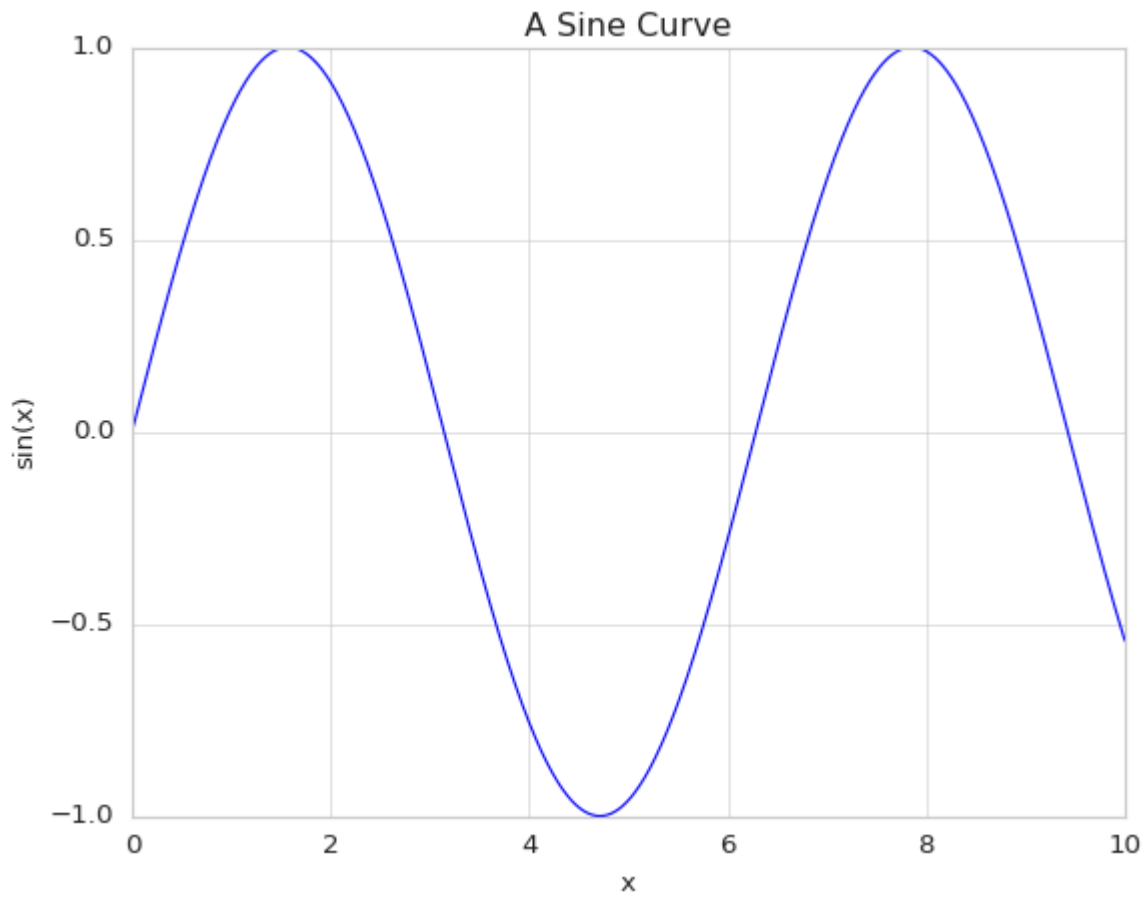


Labeling Plots

As the last piece of this section, we'll briefly look at the labeling of plots: titles, axis labels, and simple legends.

Titles and axis labels are the simplest such labels—there are methods that can be used to quickly set them:

```
plt.plot(x, np.sin(x))
plt.title("A Sine Curve")
plt.xlabel("x")
plt.ylabel("sin(x)");
```



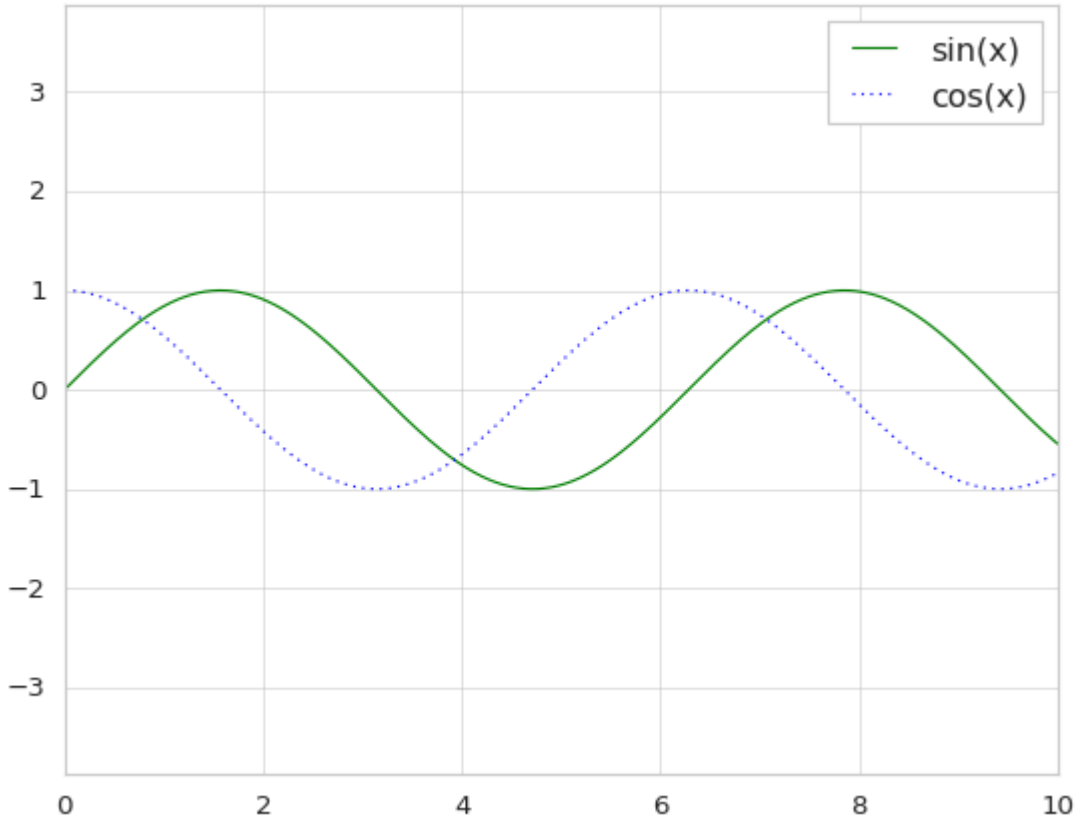
When multiple lines are being shown within a single axes, it can be useful to create a plot legend that labels each line type.

Again, Matplotlib has a built-in way of quickly creating such a legend. It is done via the (you guessed it) **plt.legend()** method.

Though there are several valid ways of using this, I find it easiest to specify the label of each line using the label keyword of the plot function:

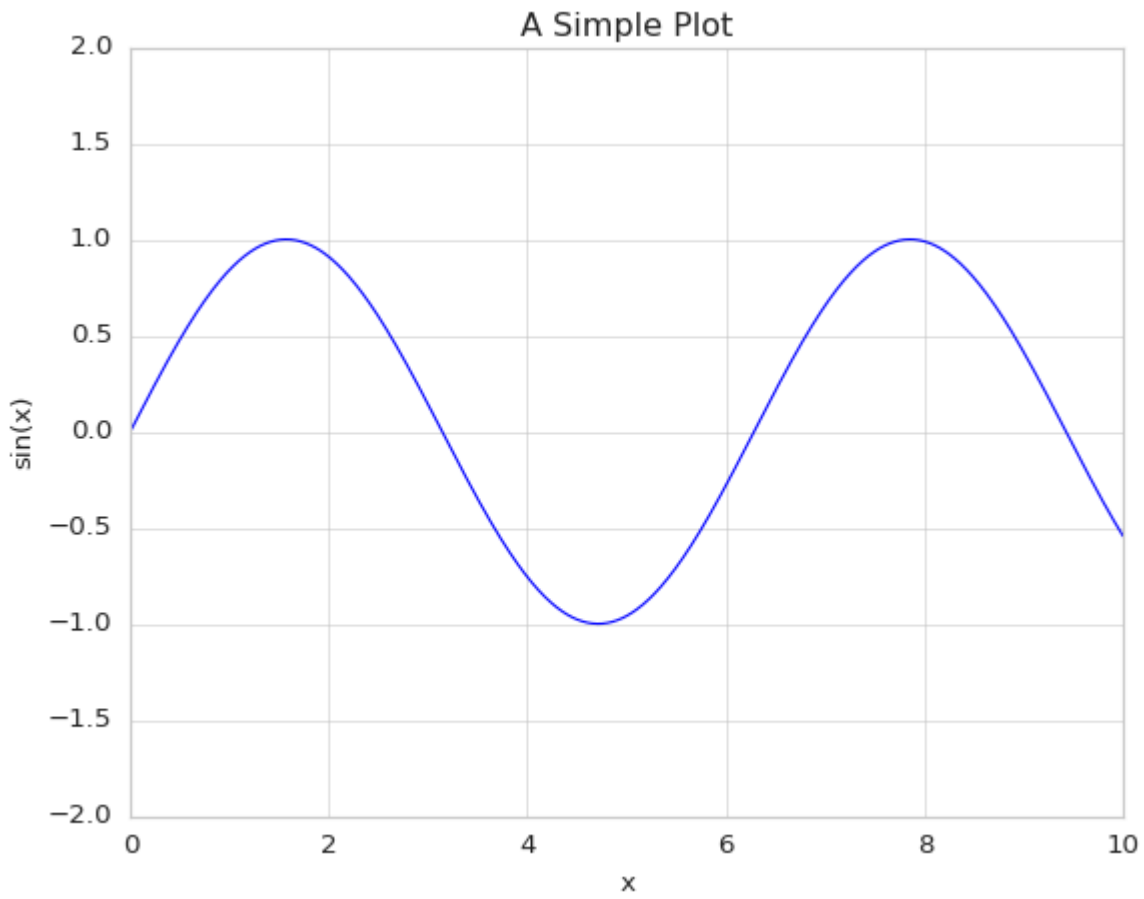
```
plt.plot(x, np.sin(x), '-g', label='sin(x)')
plt.plot(x, np.cos(x), ':b', label='cos(x)')
plt.axis('equal')

plt.legend();
```



In the object-oriented interface to plotting, rather than calling these functions individually, it is often more convenient to use the ax.set() method to set all these properties at once:

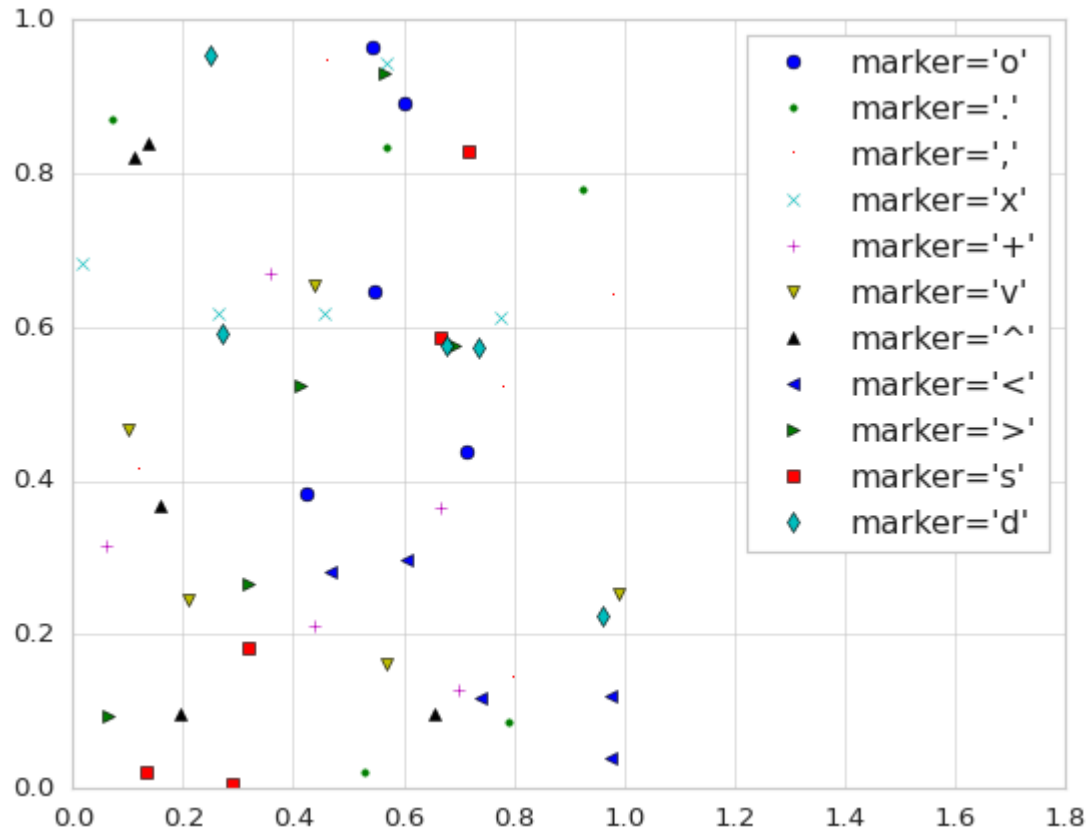
```
ax = plt.axes()
ax.plot(x, np.sin(x))
ax.set(xlim=(0, 10), ylim=(-2, 2),
       xlabel='x', ylabel='sin(x)',
       title='A Simple Plot');
```



Simple Scatter Plots

Another commonly used plot type is the simple scatter plot, a close cousin of the line plot. Instead of points being joined by line segments, here the points are represented individually with a dot, circle, or other shape.

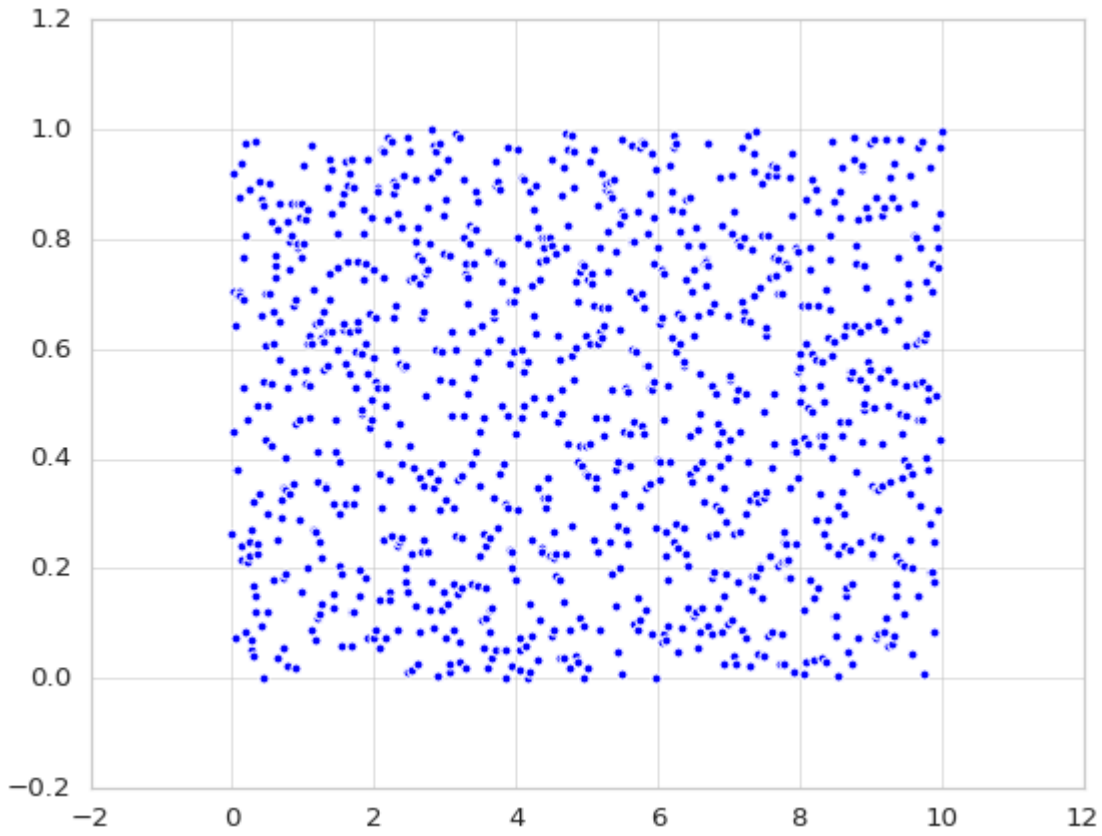
```
rng = np.random.RandomState(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.rand(5), rng.rand(5), marker,
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1)
plt.xlim(0, 1.8);
```



Scatter Plots with plt.scatter

A second, more powerful method of creating scatter plots is the plt.scatter function, which can be used very similarly to the plt.plot function:

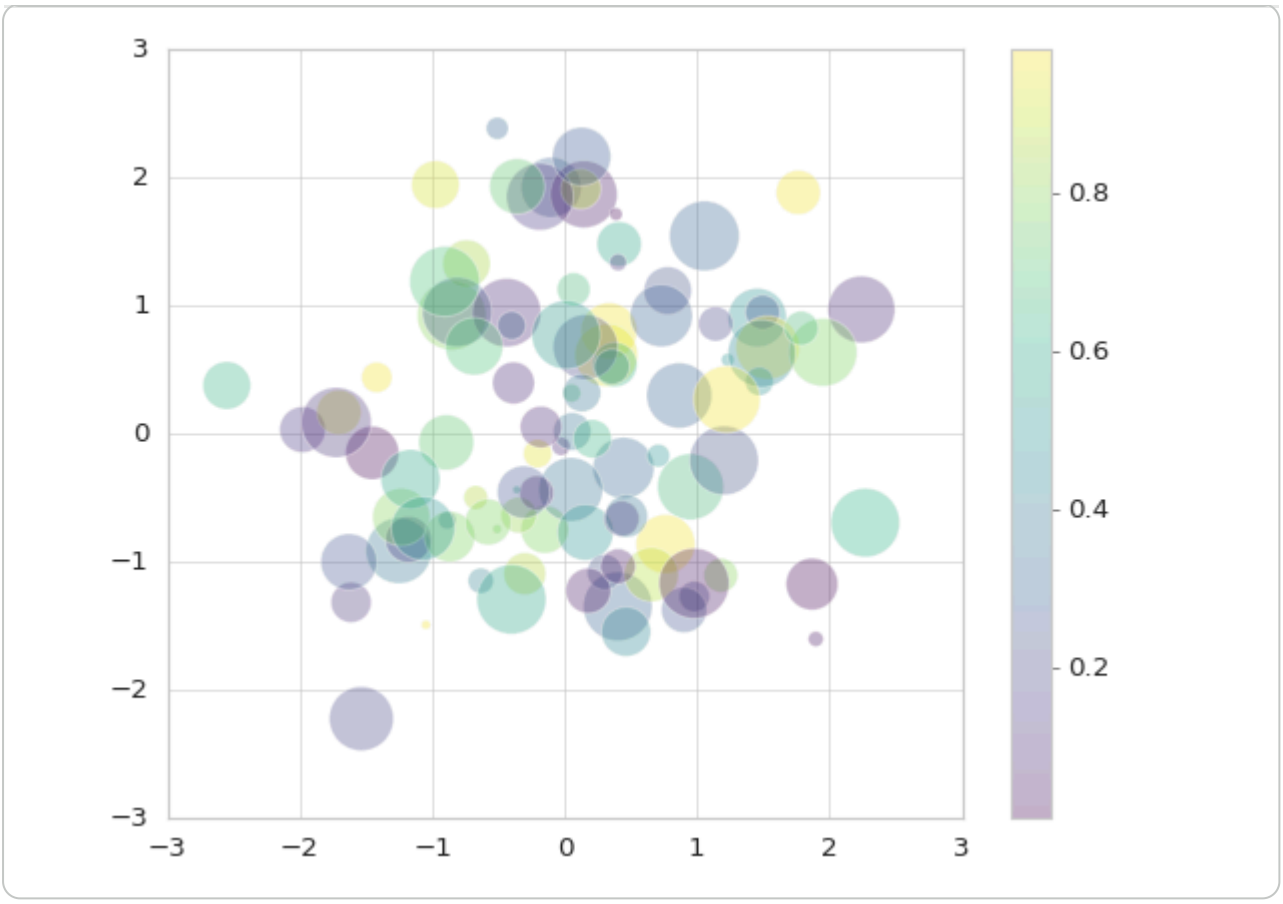
```
y = np.random.rand(len(x)) # Define y with random values matching the length
plt.scatter(x, y, marker='o');
```




```
# Consider this code for scatter plot

rng = np.random.RandomState(0)
x = rng.randn(100)
y = rng.randn(100)
colors = rng.rand(100)
sizes = 1000 * rng.rand(100)

plt.scatter(x, y, c=colors, s=sizes, alpha=0.3,
            cmap='viridis')
plt.colorbar();  # show color scale
```



Question Answer Section

1. List any 2 **python APIs** that are used for Plotting Graphs.

Ans:- Here are two prominent Python APIs used for plotting graphs:

1. Matplotlib

Matplotlib is the foundational and most widely used plotting library in Python. It provides a highly flexible and comprehensive environment for creating static, animated, and interactive visualizations. Its design philosophy is to allow users to generate plots with a few lines of code for common cases, while also providing full control over every aspect of a figure. Key characteristics include:

- **Low-level Control:** Matplotlib offers fine-grained control over all elements of a plot, including figures, axes, titles, labels, legends, line styles, colors, markers, etc.
- **Dual Interfaces:** It supports both a MATLAB-style procedural interface (`pyplot`) for quick and simple plots, and an object-oriented interface for more complex and customizable visualizations.
- **Versatility:** It can produce a wide variety of plots, including line plots, scatter plots, bar charts, histograms, 3D plots, image plots, and more.
- **Output Formats:** It supports saving plots in numerous formats, such as PNG, PDF, SVG, JPG, and EPS.

- **Integration:** It integrates well with NumPy arrays and the broader SciPy stack.

2. Seaborn

Seaborn is a high-level data visualization library based on Matplotlib. It provides a more convenient and aesthetically pleasing interface for drawing informative and attractive statistical graphics. While Matplotlib focuses on general-purpose plotting, Seaborn is specifically designed to work with pandas DataFrames and to simplify common statistical plotting tasks. Key characteristics include:

- **High-level Interface:** Seaborn offers a higher-level API compared to Matplotlib, making it easier to create complex statistical plots with fewer lines of code.
- **Statistical Focus:** It is geared towards visualizing relationships between multiple variables, distributions of data, and comparisons across different categories.
- **Aesthetics:** Seaborn comes with built-in themes and color palettes that make plots visually appealing by default.
- **Integration with Pandas:** It works seamlessly with pandas DataFrames, allowing direct plotting of columns by name.
- **Plot Types:** It excels at creating plots like heatmaps, violin plots, box plots, joint plots, pair plots, and more, which are essential for exploratory data analysis.
- **Built on Matplotlib:** Although it has its own API, Seaborn internally uses Matplotlib for rendering, meaning users can still leverage Matplotlib's customization options to fine-tune Seaborn plots.

2. Name 2 important machnisms used for accesssing Matplotlib API.

Ans:-

Two Important Mechanisms for Accessing the Matplotlib API:

Matplotlib offers two primary interfaces for creating plots, each suited for different use cases:

1. MATLAB-style Interface (using matplotlib.pyplot):

Description: This is a state-based, procedural interface primarily accessed through the matplotlib.pyplot module, typically imported as plt. It mimics the plotting commands found in MATLAB. When using this interface, Matplotlib keeps track of the 'current' figure and axes, and all plotting commands (e.g., plt.plot(), plt.title(), plt.xlabel()) apply to these active objects.

Advantages: It is very convenient and quick for generating simple plots with minimal code, making it ideal for exploratory data analysis or quick visualizations.

Disadvantages: Its stateful nature can lead to complications and lack of explicit control when dealing with more complex figures involving multiple subplots or

intricate layouts, as it becomes challenging to refer back to a specific, non-current axis.

Example Usage: plt.plot(x, y), plt.subplot(rows, cols, panel_num).

2. Object-Oriented Interface:

Description: This interface involves explicitly creating Figure and Axes objects, and then calling plotting methods directly on these objects. Instead of relying on a global state, you manage the plot elements through their respective objects.

Advantages: It provides much finer control over the entire plotting canvas and its individual elements. It is highly recommended for more complex plots, programmatic plotting, or when embedding plots within GUI applications, as it makes the code more readable, maintainable, and robust.

Disadvantages: It generally requires a few more lines of code compared to the MATLAB-style interface for basic plots.

Example Usage: fig, ax = plt.subplots(), ax.plot(x, y), ax.set_xlabel('Label').

3. Create a collection of 100 random normal numbers with parameter mean 30, sd 4 store it in object summer_temp

Ans:-

```
import numpy as np

# Create a collection of 100 random normal numbers with mean 30, sd 4
summer_temp = np.random.normal(loc=30, scale=4, size=100)

print("Generated summer_temp array (first 5 elements):\n", summer_temp[:5])
```

```
Generated summer_temp array (first 5 elements):
 [28.01535235 32.19151054 33.12224097 28.49947102 20.11063336]
```

4. Sort above set and store in object sorted_summer_temp

Ans:-

```
sorted_summer_temp = np.sort(summer_temp)

print("Sorted summer_temp array (first 5 elements):\n", sorted_summer_temp[:
```

```
Sorted summer_temp array (first 5 elements):
 [19.01205054 20.11063336 22.34215606 23.01056431 23.17049886]
```

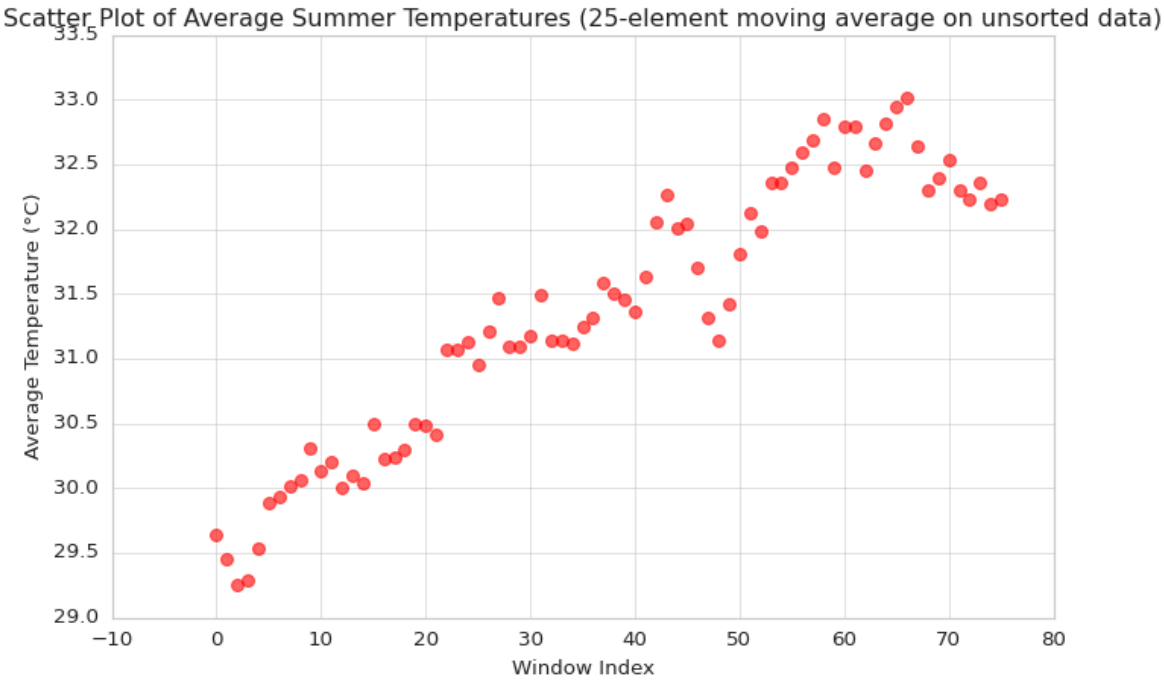
5. Store average of 25 consecutive elements from sorted_summer_temp into object x and plot it as a *line plot*. What kind of information do u get about the data?

Ans:-

```
window_size = 25
y = np.array([np.mean(summer_temp[i:i + window_size]) for i in range(len(sum

plt.figure(figsize=(10, 6))
plt.scatter(range(len(y)), y, color='red', alpha=0.6, s=50) # s is marker si
plt.title('Scatter Plot of Average Summer Temperatures (25-element moving av
plt.xlabel('Window Index')
plt.ylabel('Average Temperature (°C)')
plt.grid(True)
plt.show()

print("Calculated y (first 5 elements):\n", y[:5])
```




```
Calculated y (first 5 elements):
 [29.64409409 29.45029995 29.24987913 29.2922241  29.53746469]
```

Information from the Scatter Plot of Moving Averages (Unsorted Data):

The scatter plot of the 25-element moving average (`y`) of the original, unsorted `summer_temp` provides insights into the general temperature fluctuations and the stability of the average temperature over time (or across the sequence of generated numbers).

Key Information Inferred:

- **Central Tendency:** The points on the scatter plot will likely cluster around the mean of the original `summer_temp` dataset (which is 30 in this case). This shows that the average temperature, when considered in 25-element windows, generally hovers around the central value of the distribution.
- **Variability/Spread:** Unlike the line plot of sorted data, this plot will show the *actual* variability of the moving average in the sequence the numbers were generated. The spread of the points around the central tendency indicates how much the average temperature fluctuates from one window to the next.
- **Absence of Strong Trends:** Since the original `summer_temp` was random and unsorted, the scatter plot of its moving average `y` should not show a clear increasing or decreasing trend, unlike `x`. Instead, it should appear more like a band of points oscillating around the true mean.
- **Smoothing Effect:** Although the data is unsorted, the moving average still smooths out some of the high-frequency noise or sharp changes between individual data points. Each point in `y` represents the average of 25